```
#Ashutosh manik malewar 34(A)
from tensorflow.keras.datasets import mnist
from tensorflow.keras.utils import to_categorical
import matplotlib.pyplot as plt
import tensorflow as tf
import numpy as np
```

```
(x_train, y_train), (x_test, y_test) = mnist.load_data()
print("Original x_train shape:", x_train.shape)
print("Original y_train shape:", y_train.shape)
print("Original x_test shape:", x_test.shape)
print("Original y_test shape:", y_test.shape)
```

```
Downloading data from https://storage.googleapis.com/tensorflow/tf-keras-datasets/mnist.npz
11490434/11490434 ━━━━━━━━━━━━━━━━━━━━ 0s 0us/step
Original x_train shape: (60000, 28, 28)
Original y_train shape: (60000,)
Original x_test shape: (10000, 28, 28)
Original y_test shape: (10000,)
```

```
x_train = x_train.reshape((x_train.shape[0], 28, 28, 1))
x_test = x_test.reshape((x_test.shape[0], 28, 28, 1))
print("Reshaped x_train shape:", x_train.shape)
print("Reshaped x_test shape:", x_test.shape)
```

```
Reshaped x_train shape: (60000, 28, 28, 1)
Reshaped x_test shape: (10000, 28, 28, 1)
```

```
x_train = x_train.astype('float32') / 255.0
x_test = x_test.astype('float32') / 255.0
```

```
num_classes = 10
y_train = to_categorical(y_train, num_classes)
y_test = to_categorical(y_test, num_classes)
print("One-hot encoded y_train shape:", y_train.shape)
```

```
One-hot encoded y_train shape: (60000, 10)
```

```
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Conv2D, MaxPooling2D, Flatte
model = Sequential()
model.add(Conv2D(32, (3,3), activation='relu', input_shape=(28,
model.add(MaxPooling2D((2,2)))
model.add(Conv2D(64, (3,3), activation='relu'))
model.add(MaxPooling2D((2,2)))
model.add(Flatten())
model.add(Dense(128, activation='relu'))
model.add(Dense(num_classes, activation='softmax'))
model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/convolutional/base_conv.py:113: UserWarning: Do not pass an `input_shap
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
conv2d (Conv2D)	(None, 26, 26, 32)	320
max_pooling2d (MaxPooling2D)	(None, 13, 13, 32)	0
conv2d_1 (Conv2D)	(None, 11, 11, 64)	18,496
max_pooling2d_1 (MaxPooling2D)	(None, 5, 5, 64)	0
flatten (Flatten)	(None, 1600)	0
dense (Dense)	(None, 128)	204,928
dense_1 (Dense)	(None, 10)	1,290

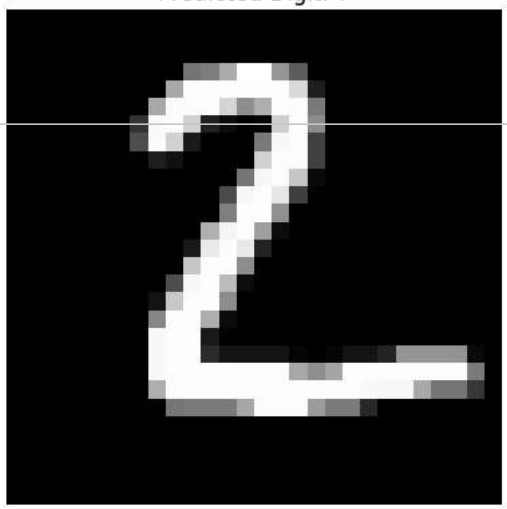
Total params: 225,034 (879.04 KB)
Trainable params: 225,034 (879.04 KB)
Non-trainable params: 0 (0.00 B)

```
model.compile(optimizer='adam', loss='categorical_crossentropy',
test_loss, test_accuracy = model.evaluate(x_test, y_test)
print("Test Accuracy:", test_accuracy)
```

```
313/313 ━━━━━━━━━━━━━━━━━━━━ 3s 9ms/step - accuracy: 0.0473 - loss: 2.3162
Test Accuracy: 0.053599998354911804
```

```
index = 0
for i in range(len(y_test)):
    if np.argmax(y_test[i]) == 2:
        index = i
        break
sample_image = x_test[index]
sample_image = np.expand_dims(sample_image, axis=0)
prediction = model.predict(sample_image)
predicted_label = np.argmax(prediction)
print("Predicted Digit:", predicted_label)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 92ms/step
Predicted Digit: 7
```

```
plt.imshow(x_test[index].reshape(28,28), cmap='gray')
plt.title("Predicted Digit: " + str(predicted_label))
plt.axis('off')
plt.show()
```

Predicted Digit: 7